

Computer Assisted Proofs in the Mathematical Sciences

Evelyn Sander

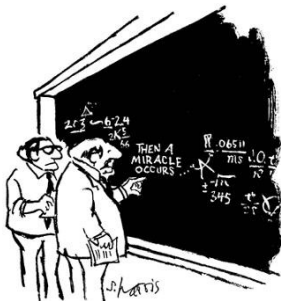


George Mason University

Lean Learning Seminar, August 26, 2026

What is a computer-assisted proof?

- A theorem is a **statement of fact**
- A **mathematical proof** consists of a validation of a theorem using **logically reasoned steps**



"I think you should be more explicit here in step two."



- What happens when there are **too many steps** to be able to look at them all?
- Have a **computer** perform the logically reasoned steps!

Neumaier's definition of a computer-assisted proof

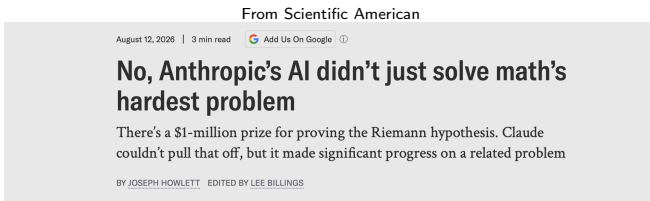
- ① Use qualitative by-hand analysis to reduce the problem to a finite number of steps
- ② Reduce each subproblem to a finite number of equations and/or inequalities
- ③ Check the equations/inequalities with a specialized computational algorithm

In some cases, the analysis and reduction steps are done after a preliminary step of AI and/or using encoded logic on a computer.

There are many different choices of specialized algorithm.

Beyond Neumaier: LLMs alone

- An critical side mention is the **iterative use of LLMs**. Effective use leads to great science.



- The **rigor remains with the human**. This is crucial, since an LLM can hallucinate wrong answers.

Three categories (my own division)

- Checkable finite number of steps: a systematic check of many finite discrete cases, computer-verified exactly
- Interval arithmetic: rigorous floating point arithmetic, combines analytical techniques with numerical methods to give rigorous statements about applied mathematical models
- Formalization: automated reasoning, to create a proof. Software such as Lean combines built-in logic and a library of existing mathematical knowledge
Especially powerful when combined with AI tools: Within the last year, this combination has led to proofs of a number of long-standing conjectures in pure math.

Part I: Checkable Finite Number of Steps

Exact checking in finitely many steps

A simple example: checking with exact arithmetic

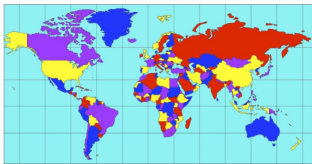
- Prove that 109 is prime
- Theoretical approach: the Sieve of Eratosthenes systematically finds all primes between 2 and n
- Using $n = \lceil \sqrt{109} \rceil = 11$, the sieve lets us verify that 109 is prime

Why can a computer perform this proof?

- 1 It is highly algorithmic
- 2 Computers perform exact arithmetic within the integers
- 3 It involves a finite number of steps

History: checkable finite number of steps

- 1976, Appel & Haken: the **four color theorem** – reduced to 1476 cases checked by computer



- 1998, Hales: proved the **Kepler conjecture** on optimal sphere packing



Part II: Interval Arithmetic

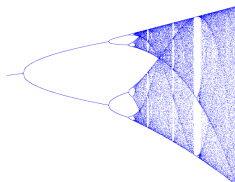
Rigorous methods for floating point arithmetic

History: interval arithmetic (i)

- 3rd century BC,
Archimedes: interval
approximation of π



- 1931, Rosalind Young: Rounding errors for floating point arithmetic
- 1958 & 1966, Ramon Moore: Computer interval arithmetic – automated error analysis
- 1982, Oscar Lanford: proved the Feigenbaum conjectures on universality of period-doubling



Interval arithmetic: working with inequalities

Prove:

$$9 < \pi^2 < 10$$

More generally: find π^2 as accurately as possible

- ① A classical formula gives $\pi^2 = 6 \sum_{n=1}^{\infty} \frac{1}{n^2}$.
- ② **Computer** Find an interval containing $\sum_{n=1}^N \frac{1}{n^2}$:
Use interval arithmetic software.
- ③ **Analysis** Find an interval containing the tail of the sum.
Use the integral test.

Interval arithmetic: a measurement example

- A person weighs 80 ± 0.5 kg, and is height 1.8 ± 0.005 meters.
- $\text{BMI} = \text{weight}/\text{height}^2 = (79.5, 80.5)/(1.7950, 1.805)^2$
- Using the interval arithmetic package Intlab within Matlab

```
>> height = infsup (79.5,80.5)
intval height =
[ 79.5000, 80.5000]
>> weight = infsup(1.795,1.805)
intval weight =
[ 1.7949, 1.8051]
>> height/weight^2
intval ans =
[ 24.4012, 24.9843]
```

Interval arithmetic: a measurement example

- With little extra effort, we can estimate derivative values using [automatic differentiation](#)
- What is the [derivative of BMI as a function of weight](#) at the point we calculated?

```
gw = gradientinit (weight)
intval gradient value gw.x =
[ 1.7949, 1.8051]
intval gradient derivative(s) gw.dx =
[ 1.0000, 1.0000]
>> height/gw^2
intval gradient value ans.x =
[ 24.4012, 24.9843]
intval gradient derivative(s) ans.dx =
[ -27.9928, -26.8876]
```

Interval arithmetic: a measurement example

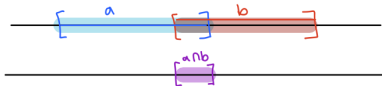
- With little extra effort, we can estimate derivative values using [automatic differentiation](#)
- What is the [derivative of BMI as a two-variable function](#)?

```
>> gw = gradientinit([height; weight])
intval gradient value gw.x =
[ 79.5000, 80.5000]
[ 1.7949, 1.8051]
intval gradient derivative(s) gw.dx =
[ 1.0000, 1.0000] [ 0.0000, 0.0000]
[ 0.0000, 0.0000] [ 1.0000, 1.0000]
>> gw(1)/gw(2)^2
intval gradient value ans.x =
[ 24.4012, 24.9843]
intval gradient derivative(s) ans.dx =
[ 0.3035, 0.3139] [ -27.9928, -26.8876]
```

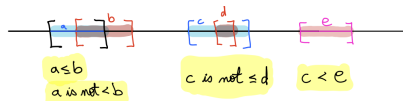
Interval arithmetic: the basics

- Interval arithmetic is arithmetic for **inequalities**: keep track of a whole set of possible values at once

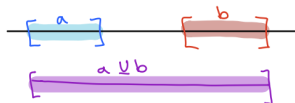
$$[\underline{a}, \bar{a}] = \{a \in \mathbb{R} : \underline{a} \leq a \leq \bar{a}\}$$
- Intersection** $[\underline{a}, \bar{a}] \cap [\underline{b}, \bar{b}]$ If two people measure the same quantity and get results $[\underline{a}, \bar{a}]$ and $[\underline{b}, \bar{b}]$, they can both be right precisely when the intersection is nonempty.



- Intervals are **partially ordered**



- Interval hull**: the smallest interval containing $[\underline{a}, \bar{a}] \cup [\underline{b}, \bar{b}]$



Interval extensions and the Fundamental Theorem

- An **extension** of $f : \mathbb{R} \rightarrow \mathbb{R}$ is a function F on intervals with $F([x, x]) = [f(x), f(x)]$
- **Arithmetic extensions**: for an operator $\circ \in \{+, -, \times, \div\}$,

$$a \circ b := \{x \circ y : x \in a, y \in b\}$$

- F is **inclusion isotonic** if $a \subseteq b \implies F(a) \subseteq F(b)$; all rational functions built from $+, -, \times, \div$ satisfy this

Fundamental Theorem of Interval Analysis: if F is an inclusion isotonic extension of f , then for all intervals a ,

$$\{f(x) : x \in a\} \subseteq F(a).$$

Our interval function evaluation is guaranteed to **contain** all the true values – though it may contain more.

Beyond arithmetic, beyond one dimension

Interval arithmetic extends to

- Transcendental functions (trig functions, logs, ...)
- Rigorous root finding (interval Newton's method)
- Higher dimensions
- Linear algebra (even eigenvalues and eigenvectors)

Pitfalls

Overestimation: In general,

$$a - a \neq \{0\}$$

due to **interval dependency**: naive evaluation only guarantees $x_1 - x_2 \in a - a$ for $x_1, x_2 \in a$, not $x_1 = x_2$.

Non-Associativity: Let

$$A = \begin{pmatrix} 1 & [0, 1] \\ 1 & -1 \end{pmatrix}$$

Multiplication is **non-associative**!

$$(A \cdot A) \cdot A \neq A \cdot (A \cdot A)$$

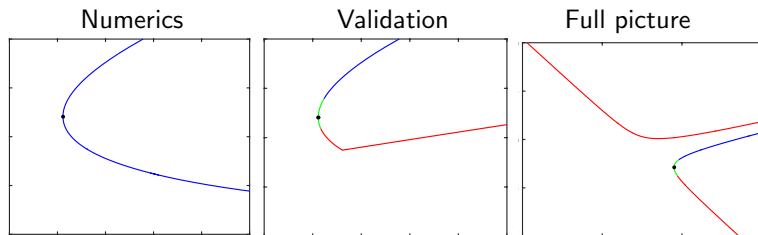
Applications of these methods

Leveraging existing theoretical and numerical methods gives interval bounds for infinite series, integrals, ordinary differential equation solutions, partial differential equation solutions, bifurcations, chaos, such as

- Global dynamics, topological entropy, invariant manifolds
- n -body choreography in celestial mechanics
- Traveling waves, pattern-forming PDE (hexagons, rolls)
- Progress toward blow-up solutions for Navier–Stokes equations (Clay prize problem)
- Rigorous bifurcation theory (Sander, Wanner)

Validation catches what numerics miss

Discrete Allen–Cahn model: ODE in 10 dimensions
(Sander, Wanner)



- Pencil and paper proofs are intractable
- Validation improves error analysis: reveals fine structure that could not be found using floating point numerics

Part III: Formalization and AI

Automated reasoning

Formalization of proof

- 1996, McCune: proved the [Robbins conjecture](#) using an automated theorem prover
- 2013, Leonardo de Moura creates [Lean](#), a programming language that is also a proof assistant
- Now, [Mathlib](#), its community-built library, contains formalizations of [over 210,000 theorems](#) – and is growing fast
- Now, proofs of longstanding conjectures are coming out every month

Why learn Lean?

Enthusiastic endorsement:

- Advocated by celebrated mathematicians, including [Thomas Hales](#) (Kepler conjecture – Part II), [Kevin Buzzard](#), and [Terence Tao](#).
- **Mathematicians:** Formalized proofs already give a higher degree of validity than traditional (“bespoke”) ones. [Alex Kontorovich](#) predicts formalization winning out as the **standard of proof**. Better to learn Lean now than have it become an existential necessity later!
- **Other Scientists:** Formalization means you know that what you are getting is a result and not just an output!

What is formalization?

- A formal proof assistant (e.g. [Lean](#), Coq, Isabelle) encodes the axioms and rules of logic directly as a programming language
- A community-built library (for Lean: [Mathlib](#)) accumulates formalized definitions and theorems, so new proofs can build on existing verified results
- Every logical step must be verified directly or by calling on a pre-known theorem
- Verify the logic but no way to validate numerics using intervals. [Best would be to combine the two!](#)

What exactly is formalization

Lean is a programming language. Its building blocks are:

- **Goals**, propositions, theorems, examples: a statement which can be true or false
- **Tactics** for proving goals: library of known results, subresults developed by you: a set of known results
- **Logic** built into Lean: assess whether your tactics validate your goal

```
21
22   example (x y z : ℚ) (h1: x > y) (h2: y < z): x < z := by
23   linarith
24
25   example (x y z : ℚ) (h1: x ≤ y) (h2: y < z): x < z := by
26   linarith
27
```

A Lean example: infinitely many primes

Theorem (aka Goal): There are infinitely many primes

Blueprint:

- Assume there are finitely many primes, all less than n
- Then $n! + 1$ has a prime factor $p \leq n$
- Thus p divides both $n!$ and $n! + 1$, so p divides 1
- But no prime divides 1, so this is a contradiction

Formalization snippet built interactively, minimal AI assistance

```
theorem infinite_primes :  $\forall n : \mathbb{N}, \exists p > n, \text{Nat.Prime } p$  := by
  intro n
  obtain ⟨p, hp⟩ := primefactors (n ! + 1) (by ...)
  have pgt :  $p > n$  := by
    by_contra pnotgt
    ...
  have :  $p \mid 1$  := by simpa using Nat.dvd_sub div1 div2
  exact Nat.Prime.not_dvd_one hp.left this
exact ⟨p, pgt, hp.left⟩
```

A Lean example: $\sqrt{2}$ is irrational

Goal: $\sqrt{2}$ is irrational

Blueprint: If x, y are coprime then $(x/y)^2 \neq 2$ because

- If x^2 is even, then x is even.
- Implying y^2 is even, and thus y is even.
- 2 divides both x and y , contradicting x, y coprime.

Formalization snippet built interactively, minimal AI assistance

```
theorem two_irrational (x y : ℕ) (h : Nat.Coprime x y) :  
  x^2 ≠ 2 * y^2 := by  
  intro sqr_eq  
  have x2even : 2 ∣ x^2 := ...  
  have xeven  : 2 ∣ x   := Nat.Prime.dvd_of_dvd_pow Nat.prime_two x2even  
  have x4     : 4 ∣ x^2 := ...  
  have yeven  : 2 ∣ y   := ...  
  have : 2 ∣ Nat.gcd x y := Nat.dvd_gcd xeven yeven  
  ... -- contradicts h : Nat.Coprime x y
```

Formalization meets AI

- A formalized proof is a higher bar than traditional proof
- Lean is a large language: proofs, logic, math library
- AI tools (large language models, reinforcement-learning provers) can search for formal proof steps automatically, dramatically speeding up formalization
 - LLM-based provers built on Lean (e.g. [GoogleProof](#), [AlphaProof](#), [Aristotle](#)) have gone beyond re-proving known theorems, producing new proofs of previously unknown results
 - Large collaborative projects formalizing major results in Lean/Mathlib, notably the [First Proof](#) collaboration

Accelerated progress

AI-assisted formalization progress is accelerating

- [Sawhney, Sellke, Tao et al.](#): ~ 100 open Erdős problems resolved with LLM assistance since Oct. 2025, some with genuinely new proofs
- [Gemini / OpenAI](#): gold-medal-level scores on IMO 2025
- [OpenAI's model](#): disproves Erdős's 80-year-old unit-distance conjecture, finding a counterexample humans overlooked (2026)
- [PINNs + DeepMind](#): uncover new candidate singularities in fluid equations (Euler, porous medium), extending Hou–Chen's approach (2026)

Philosophical questions

- Mathematics consists of knowing what is true and why: **underlying structure of mathematics.**
- An LLM-produced formalized proof has mathematical rigor, but **can be unreadable by humans.**
- Eg: In 2026, Maryna Viazovska (Fields Medalist) and Sidharth Hariharan were formalizing a proof on sphere packing when Gauss by Math Inc. scooped them. **The proof is not human readable.**
- If it doesn't add insight, **does an unreadable AI-Lean proof even fit the definition of a proof?**
- Maybe we have to divide math into two pieces: (1) What is true and (2) why – **like other sciences!**

Why does any of this matter beyond mathematics?

- Interval arithmetic gives rigorous, quantitative guarantees for models used throughout science and engineering – wherever floating point computation drives a scientific conclusion
- Formalized, AI-assisted proof offers machine-checkable certainty even for arguments too long or intricate to check by hand
- Both approaches are actively used on open problems of broad scientific interest
- They force us to think about the actual meaning of mathematical proof

Summary

- Checkable finite number of steps, interval arithmetic, and formalization let us push rigorous mathematics past the limits of by-hand analysis
- Interval arithmetic rigorously tracks numerical error, turning floating point computation into a source of proof
- Formalization, especially combined with AI, is rapidly automating the verification – and even discovery – of long and intricate proofs
- Together, these tools are reshaping what counts as a proof, and are increasingly relevant well beyond pure mathematics
- Math is not just for mathematicians anymore!

Further reading

References and instructions:

<https://github.com/esander1789/LeanLearningSeminar>

Thank you!